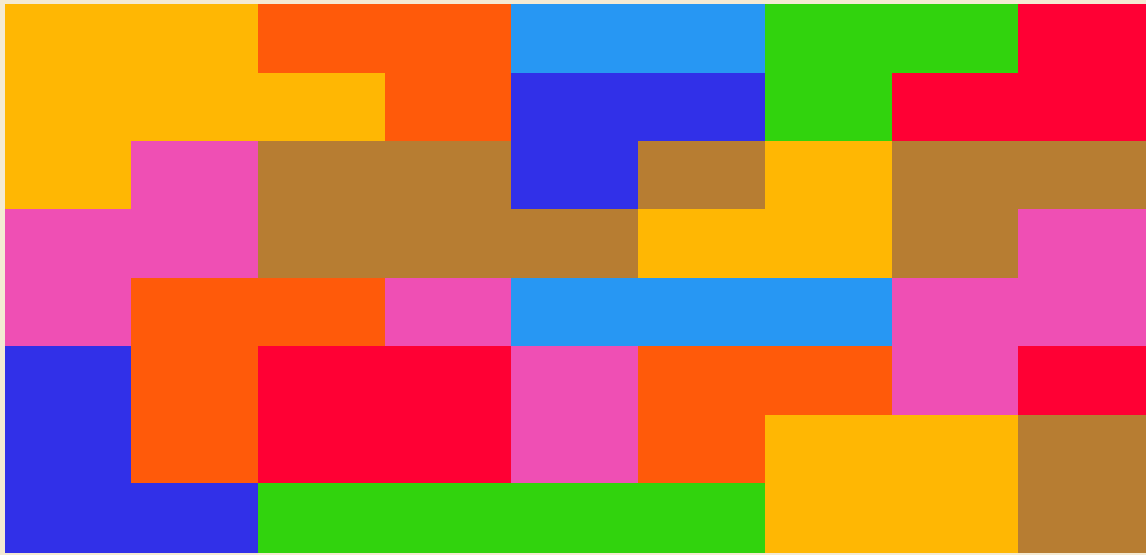




DRONE CAMP

DAY 2

Program a DJI Tello with Python.

PYTHON + EASYTELLO + DRONE COMMANDS

MISSION

Turn Python into a flight plan

easyTello gives Python a set of methods for communicating with a DJI Tello. But what is your program actually doing? It creates one object that represents the drone connection, then calls methods on that object in a specific order. Values inside the parentheses describe details such as distance, angle, or time. easyTello takes each Python method call, translates it into a command message the Tello understands, sends that message across the local network, and waits for the drone to reply.

Core pattern

`object.method(argument)` means: ask this object to perform one action using the value inside the parentheses.

Follow one command

COMMAND FLOW

```
Python code
|
v
drone.forward(50)
|
v
easyTello sends "forward 50"
|
v
Tello moves and replies
```

What you will practice

- objects: one Python variable represents the drone
- methods: named actions such as `takeoff()`, `forward()`, and `land()`
- arguments: values that control distance or angle
- sequence: commands execute in the order you write them
- loops: repeated flight actions can be expressed once

Flight rule

A working program is never automatic permission to launch. Wait for the instructor to clear the flight area.

CHECKPOINT 01

Install, connect, and create the drone object

POWERSHELL - INSTALL THE LIBRARY

```
python -m pip install easytello
python -m pip install opencv-python
```

Before a flight command can work, the computer and the drone must first be able to exchange messages. Connecting to the Tello Wi-Fi network places the computer on the drone's local network. When `tello.Tello()` runs, the library opens a UDP socket, prepares the drone's network address, starts a background listener for replies, and sends the command that places the Tello into SDK command mode. Only after that setup can later methods send movement instructions or request sensor information.

FILE LEVEL - CREATE ONE DRONE OBJECT

```
from easytello import tello

drone = tello.Tello()

battery = drone.get_battery()
print("Battery:", battery)
```

Read the code

- `from easytello import tello` brings the library module into the file.
- `tello.Tello()` creates the object and enters command mode.
- `drone` is the variable used for every later instruction.
- `get_battery()` is a query: it asks for information and returns a response.

First checkpoint

Run this before any flight code. A battery response shows that Python and the drone are communicating.

CHECKPOINT 02

Write the smallest complete flight

COMPLETE PROGRAM - TAKEOFF -> WAIT -> LAND

```
from easytello import tello

drone = tello.Tello()

drone.takeoff()
drone.wait(2)
drone.land()

drone.close()
```

Why this is a complete program

What makes this a complete flight rather than a random collection of commands? It has a beginning, a controlled pause, an ending, and cleanup. Creating the object places the Tello into command mode. `takeoff()` requests liftoff. `wait(2)` pauses the Python program before it sends the next instruction, giving the drone time to remain in place. `land()` requests a normal landing, and `close()` closes the computer's local communication socket after the final response has been handled.

Sequence is behavior

Python reads these statements from top to bottom, so the order itself describes the flight. Suppose `land()` appeared before `takeoff()`. Both method names would still be spelled correctly, but the sequence would no longer make sense. This is an important robotics idea: a program can contain individually valid instructions and still produce the wrong behavior because those instructions were arranged incorrectly. Before launching, trace the program one line at a time and state what the drone should be doing after each line.

Before launch	Drone centered, propellers clear, people behind the boundary, battery checked, instructor watching.
Normal ending	Use <code>land()</code> for ordinary operation. <code>emergency()</code> is reserved for an instructor-directed emergency.

CHECKPOINT 03

Control distance and rotation with arguments

FILE LEVEL - VARIABLES CONTROL THE FLIGHT

```
from easytello import tello

drone = tello.Tello()

MOVE_DISTANCE = 50
TURN_ANGLE = 90

drone.takeoff()
drone.up(50)
drone.forward(MOVE_DISTANCE)
drone.cw(TURN_ANGLE)
drone.wait(1)
drone.land()
drone.close()
```

Arguments make methods adjustable

An argument is an input that changes how a method behaves. The easyTello developers could provide a forward() method, but they could not know how far every future program should travel. That decision belongs to the programmer. forward(50) and forward(100) therefore use the same method with different arguments. Constants such as MOVE_DISTANCE and TURN_ANGLE give those numbers meaningful names. If the route must change later, you can revise one constant instead of searching through the entire file for every repeated number.

Useful movement methods

- up(distance) and down(distance) change height
- forward(distance) and back(distance) change front-to-back position
- left(distance) and right(distance) move sideways
- cw(degrees) and ccw(degrees) rotate clockwise or counterclockwise

Notice that movement and rotation change different parts of the drone's state. forward() changes position without intentionally changing the direction the drone faces. cw() changes direction without intentionally traveling to a new location. A useful trace therefore keeps track of both position and heading. After every line, ask two separate questions: where should the drone be now, and which way should its front be pointing?

Predict first

Before running, trace every line and describe the expected position and direction after each command.

CHECKPOINT 04

Use a loop to build a square flight path

LOOP SCOPE - REPEAT ONE SIDE FOUR TIMES

```
from easytello import tello

drone = tello.Tello()

MOVE_DISTANCE = 50
TURN_ANGLE = 90

drone.takeoff()
drone.up(50)

for side in range(4):
    drone.forward(MOVE_DISTANCE)
    drone.cw(TURN_ANGLE)
    drone.wait(1)

drone.land()
drone.close()
```

What the loop represents

Why does this loop describe a square? Each cycle contains the same two-step pattern: move forward once, then rotate ninety degrees. `range(4)` produces four cycles, one for each side. During those cycles, the variable `side` receives the values 0, 1, 2, and 3. This program does not need to use those values; Python still updates the variable as it counts. Four ninety-degree turns total 360 degrees, so the final heading should match the starting heading, even though real flight may include a small amount of drift.

Why loops matter in robotics

You could write the forward-and-turn pair four separate times, and the drone might still follow the same route. So why use a loop? Repetition creates more places for typing mistakes: one distance might change, one turn might be omitted, or one line might be placed in the wrong order. A loop states the repeated rule once and lets Python perform it several times. That makes the route shorter, but more importantly, it makes the programmer's intention easier to inspect and revise.

Debug on paper

Draw the starting arrow, then draw four forward arrows and four right turns. The final arrow should face the starting direction.

UNDER THE HOOD

What easyTello does for your Python code

LIBRARY SOURCE - METHODS BUILD COMMAND STRINGS

```
def forward(self, dist: int):
    self.send_command(
        "forward {}".format(dist)
    )

def get_battery(self):
    self.send_command("battery?", True)
    return self.log[-1].get_response()
```

The method is a translator

When your program calls `drone.forward(50)`, Python does not directly control the motors. The forward method first builds the text command "forward 50". `send_command()` then converts that text into bytes, sends those bytes to the Tello network address, and creates a Stats entry in the command log. While the main code waits, a separate response thread listens for data arriving from the drone. The response is attached to the newest log entry, allowing the program to report success, return query data, or stop waiting after a timeout.

Control commands and queries

- control command: asks the drone to perform an action, such as takeoff or forward
- query: asks the drone to report information, such as battery, height, or temperature
- response thread: listens in the background so replies can arrive while the main program continues
- log: connects each outbound command with the response that follows

Technical detail

In the provided class, commands and replies use UDP port 8889, and the default drone address is 192.168.10.1.

RECAP

What you learned to control

PROGRAM RECAP

```
1. import the library
2. create one Tello object
3. query the battery
4. call methods in flight order
5. pass distance and angle arguments
6. use a loop for repeated movement
7. land and close the connection
```

The reusable robotics pattern

Connect -> check -> command -> wait -> observe -> land. This sequence is useful because it separates preparation from movement and movement from shutdown. A readable flight program should let another developer answer three questions before launch: Where should the drone move? Which direction should it face afterward? What instruction safely ends the route? If those answers are unclear from the code, simplify the sequence, rename the variables, or test a smaller section first.

Key terms, stated precisely

The object is the Python value stored in drone. A method is a function attached to that object, such as `takeoff()` or `get_battery()`. An argument is a value supplied to a method, such as 50 in `forward(50)`. A query requests information and returns a response. A loop repeats a defined block of code. These words describe different parts of the program, so using the correct term makes debugging and discussion much clearer.

Common failures

- No response: confirm the computer is connected to the Tello Wi-Fi network.
- Module not found: install `easytello` inside the active Python environment.
- Connection timed out: confirm the drone is powered, nearby, and not controlled by another device.
- Unexpected path: inspect command order, constants, loop indentation, and rotation direction.
- Program stops before landing: keep the plan small and test one new movement at a time.

Final check

Explain what object, method, argument, loop, query, and response mean using one line from your own program.

Library reference

Guide based on the `easyTello` PyPI usage examples and the Tello class source supplied for Drone Camp.